Gradient Descent and Its Variants

Machine Learning-2 — Lecture 4, Indian Institute of Technology Jodhpur

Formatted by: Heisenberg 😎

Index

1. **Recap — Where We Left Off** — *the "previously on ML-2" montage.*
2. **The Gradient Vector and Directional Derivative** — *which way is "uphill", precisely speaking.*
3. **The Gradient Descent Algorithm** — *walk downhill, one small step at a time.*
4. **Why Does Gradient Descent Actually Work? (Taylor Series Proof)** — *the math that gives you permission to trust the algorithm.*
5. **Worked Example 1 — Single Variable,** $f(x) = x^2$ — *how learning rate can make or break you.*
6. **Worked Example 2 — Two Variables,** $f(x_1, x_2) = x_1^2 + 2x_2^2$ — *every matrix multiplication, shown in full, no shortcuts.*
7. **Gradient Ascent for Logistic Regression** — *same idea, uphill this time.*
8. **The Computational Bottleneck — Why We Need Something Better** — *when "sum over all your data" stops being funny.*
9. **Stochastic Gradient Descent (SGD)** — *one data point to rule the gradient (approximately).*
10. **The Dynamic Learning Rate Trick** — *slow down as you approach the finish line.*
11. **The Dimensional Scaling Problem** — *when one direction crawls and another explodes.*
12. **Momentum — "Don't Lose the Momentum"** — *remembering your last few steps so you don't zig-zag forever.*
13. **AdaGrad — Adaptive Gradient** — *a personal learning rate for every parameter.*
14. **RMSProp** — *AdaGrad, but it doesn't hold a permanent grudge.*
15. **Adam — Adaptive Moment Estimation** — *momentum and RMSProp, happily married.*
16. **Summary — Comparing All the Variants**
17. **Practice Questions**

---

1. Recap — Where We Left Off

In the previous lecture, the setting for Gradient Descent was established: we are given a scalar-valued function f that consumes a vector and produces a single number—for instance, a **loss function**, where the input is a data point of dimension n (fed to a neural network) and the output is compared against a label (discrete for classification, continuous for regression). Once this loss is defined, the task becomes finding the parameters that **minimize** it.

To do this, we defined the function and its **gradient vector**, which stacks all the partial derivatives with respect to every variable. The variables $x_1, \ldots, x_n$ are general placeholders—in practice, these are the weights of a neural network or the learnable parameters of any ML model.

We saw that Gradient Descent minimizes a function by:

1. Initializing a solution $x^{(0)}$ and a step size (learning rate) α.
2. Repeatedly moving from the current solution to the next one **in the direction of the negative gradient**, scaled by α.

For a **minimization** problem, the step is **subtracted** (move against the gradient). For a **maximization** problem, the step is **added** (move along the gradient)—this is called **Gradient Ascent**.

We also observed that the **rate of convergence** depends heavily on α: too small, and convergence is painfully slow; too large, and the solution can diverge to an undefined value entirely. The safe default is to start with a small learning rate—it won't diverge, but it will take longer.

> **Note for Review:** The single biggest lever in Gradient Descent is the learning rate α. Every variant discussed in this lecture (SGD, Momentum, AdaGrad, RMSProp, Adam) is fundamentally about finding smarter ways to *set or adapt* α—either to converge faster or to stop it from blowing up.

Today's lecture builds on this by:

- Generalizing Gradient Descent to the multivariable case with a formal proof.
- Addressing the **computational cost** of computing gradients over huge datasets.
- Addressing the problem of **different gradient magnitudes along different directions**.

2. The Gradient Vector and Directional Derivative

Let $f : \mathrm{R}^n \to \mathrm{R}$ be a scalar function. The **gradient** of $f(x_1, x_2, \ldots, x_n)$ is defined as

$$\nabla f = \begin{bmatrix} \dfrac{\partial f}{\partial x_1} \\ \dfrac{\partial f}{\partial x_2} \\ \vdots \\ \dfrac{\partial f}{\partial x_n} \end{bmatrix}.$$

Geometrically, for a level curve $f(x_1, x_2) = c$, the gradient ∇f is a vector that is **perpendicular** to the level curve at every point, pointing toward increasing values of f.

The **directional derivative** of f along a unit direction vector v, at a point x, tells us the instantaneous rate of change of f if we move along v:

$$D_v f(x) = v^\top \nabla f(x) = \cos\theta \, \|\nabla f(x)\|,$$

where θ is the angle between v and $\nabla f(x)$.

Key Fact

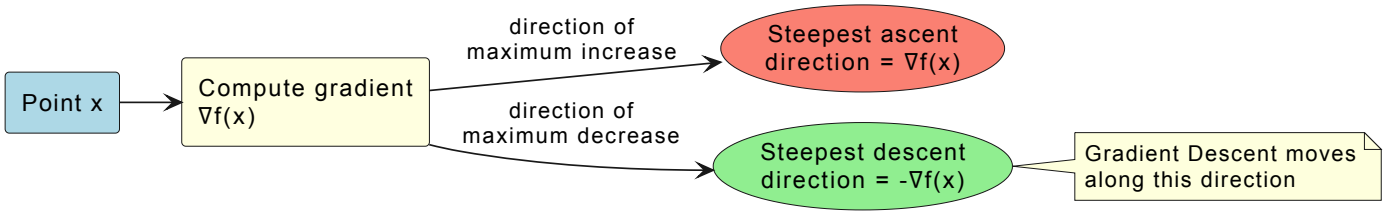
The directional derivative is **maximized** when v points in exactly the same direction as $\nabla f(x)$ (that is, $\theta = 0^\circ$):

$$\max_{\|v\|=1} v^\top \nabla f(x) = \|\nabla f(x)\|,$$

which occurs when

$$v = \frac{\nabla f(x)}{\|\nabla f(x)\|}.$$

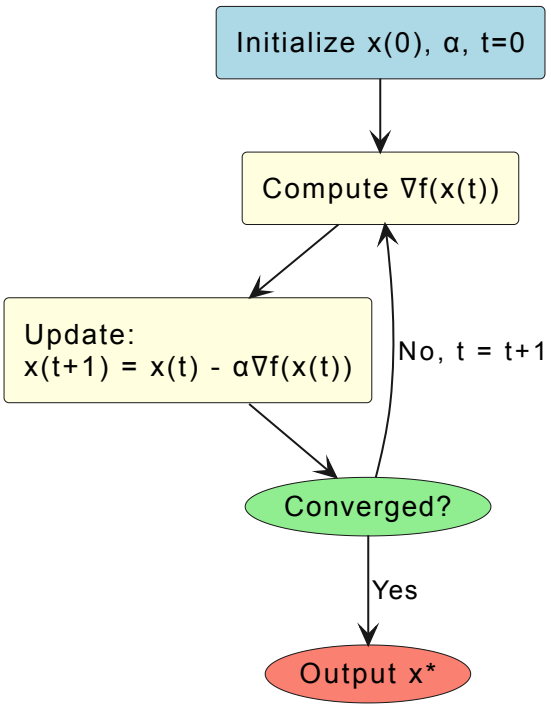
This single fact is the entire justification for gradient-based optimization: **the gradient points in the direction of steepest ascent**, so moving opposite to it is, locally, the fastest way to decrease f.



The Gradient Descent Algorithm

Let $\nabla f(x)$ be the gradient of f at point x.

> **Algorithm: Gradient Descent**
>
> **Input:** Initial point $x^{(0)}$, learning rate α
>
> 1. Initialize $x^{(0)}$ and choose a step size α.
> 2. Set $t \leftarrow 0$.
> 3. **While** not converged **do**
> 4. Update the parameters:
>
> $$x^{(t+1)} = x^{(t)} - \alpha \nabla f(x^{(t)})$$
>
> 5. Increment the iteration counter:
>
> $$t \leftarrow t + 1$$
>
> 6. **End while**



Every subsequent variant discussed in this lecture (SGD, Momentum, AdaGrad, RMSProp, Adam) is a modification of **line 4** of this algorithm — either changing what gradient is used, or changing how the step size α behaves.

4. Why Does Gradient Descent Actually Work? (Taylor Series Proof)

Deriving the Gradient Descent Update Rule (Taylor Series)

So far the update rule was stated without proof. Here is where it comes from, generalized to **n dimensions** using a first-order **Taylor series expansion**.

Let $f : \mathrm{R}^n \to \mathrm{R}$, and consider a point x perturbed by a very small vector ϵ (every component of ϵ is small). By the definition of the multivariate Taylor series,

$$f(x + \epsilon) = f(x) + \frac{\epsilon^\top \nabla f}{1!} + \frac{1}{2!}\epsilon^\top \nabla^2 f\, \epsilon + \cdots$$

Here $\nabla^2 f$ is the **Hessian** (the matrix of second-order partial derivatives), and there are infinitely many further higher-order terms—we only need the first-order term for this argument.

Goal

We want to choose ϵ such that

$$f(x + \epsilon) < f(x),$$

i.e., we want to move to a point with a **smaller function value**.

Choice of ϵ

Pick

$$\epsilon = -\eta\, \nabla f, \qquad \eta > 0,$$

where η is the learning rate, always chosen to be positive.

Substituting this ϵ into the Taylor expansion

Dropping the second-order and higher-order terms (since ϵ is very small),

$$f(x - \eta \nabla f) = f(x) - \eta\, \frac{\nabla f^\top \nabla f}{1!}.$$

Since $1! = 1$,

$$f(x - \eta \nabla f) = f(x) - \eta\, \nabla f^\top \nabla f.$$

Using the identity

$$\nabla f^\top \nabla f = \|\nabla f\|_2^2,$$

we obtain

$$f(x - \eta \nabla f) = f(x) - \eta\, \|\nabla f\|_2^2.$$

Why does this decrease the function?

Notice that

$$\nabla f^\top \nabla f = \|\nabla f\|_2^2$$

is exactly the **squared 2-norm of the gradient**, which is **always non-negative** because it is a sum of squares.

Since $\eta > 0$, the quantity

$$\eta\, \|\nabla f\|_2^2$$

is also non-negative and is being **subtracted** from $f(x)$.

Therefore,

$$f(x - \eta \nabla f) < f(x),$$

whenever

$$\nabla f \not\equiv 0.$$

Interpretation

If we move from the current point x to the new point

$$x - \eta \nabla f,$$

the function value decreases (as long as $\nabla f \not\equiv 0$). This means the new point is a **better solution** than the current one under the first-order approximation, so repeating this update continues moving toward a minimum.

The Gradient Descent Mapping

The parameter update is

$$x \;\longrightarrow\; x - \eta \nabla f(x).$$

The corresponding function values evolve as

$$f(x) \;\longrightarrow\; f(x - \eta \nabla f(x))\,,$$

with

$$f(x - \eta \nabla f(x)) < f(x).$$

This is exactly the iterative update rule of **Gradient Descent**—now derived rather than assumed.

> 📝 **Note for Review:** This proof only guarantees a *decrease* for a **sufficiently small** \eta — because we dropped the second-order (Hessian) term, which is only negligible when \epsilon is small. This is precisely *why* an overly large learning rate can cause divergence: the dropped higher-order terms are no longer negligible, and the "guaranteed decrease" argument breaks down.

---

5. Worked Example 1 — Single Variable, f(x) = x^2

Example: Effect of Learning Rate in Gradient Descent

Consider $f(x) = x^2$, and we want to find the point of minima using Gradient Descent starting at $x_0 = 10$.

Analytically we know the minimum is at $x^* = 0$, since

$$f'(x) = 2x = 0 \;\Rightarrow\; x = 0.$$

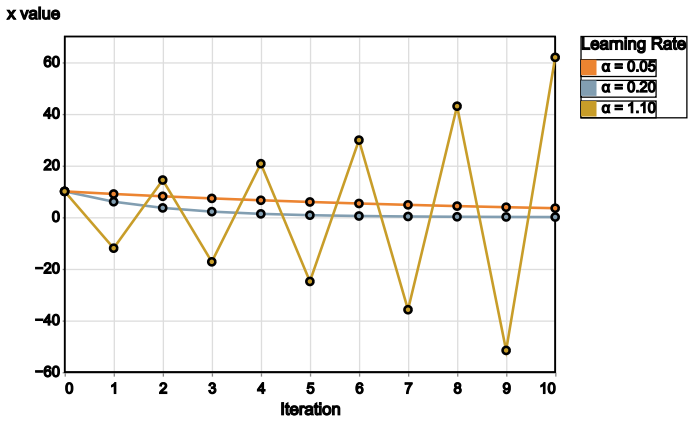
The update rule for this one-dimensional case is simply

$$x^{(t+1)} = x^{(t)} - \alpha \cdot 2x^{(t)}.$$

Three Different Learning Rates

Learning Rate (α)	Behavior
$\alpha = 0.20$	Converges smoothly and quickly to $x = 0$ in roughly 10 iterations.
$\alpha = 0.05$	Converges, but far more slowly—after the same 10 iterations, the solution is still noticeably far from 0.
$\alpha = 1.10$	**Diverges.** Starting from $f(10) = 100$, the solution overshoots the minimum on every step, and the function value explodes—reaching roughly 4000 within about 10 iterations, oscillating wildly with further iterations.

Visualization of Different Learning Rates



The $\alpha = 1.10$ series follows the exact update

$$x^{(t+1)} = x^{(t)} - 1.10 \cdot 2x^{(t)} = x^{(t)}(1 - 2.2) = -1.2\,x^{(t)}.$$

So the magnitude grows by a factor of **1.2** every iteration while the sign flips each step—a textbook divergence pattern.

Takeaway

Correctly choosing the learning rate is the single most important practical decision in Gradient Descent:

- **Small α:** stable but slow convergence.
- **Well-chosen α:** fast convergence.
- **Large α:** overshooting and divergence.

---

6. Worked Example 2 — Two Variables, $f(x_1, x_2) = x_1^2 + 2x_2^2$

This is the full multivariable generalization, worked out with every matrix computation shown explicitly — nothing skipped.

Function:

$$f(x_1, x_2) = x_1^2 + 2x_2^2$$

Gradient:

$$\nabla f(x_1, x_2) = \begin{bmatrix} \dfrac{\partial f}{\partial x_1} \\ \dfrac{\partial f}{\partial x_2} \end{bmatrix} = \begin{bmatrix} 2x_1 \\ 4x_2 \end{bmatrix}$$

Worked Example: Gradient Descent in Two Dimensions

Optimal point (set gradient to zero):

$$2x_1 = 0, \qquad 4x_2 = 0 \;\Rightarrow\; x_1^* = 0,\; x_2^* = 0.$$

So the global minimum is at

$$x^* = (0, 0).$$

Setup

- Initial solution:

$$x^{(0)} = \begin{bmatrix} -5 \\ -2 \end{bmatrix}$$

- Learning rate:

$$\alpha = 0.1$$

- Update rule:

$$x^{(t+1)}$$

Iteration 1: Computing $x^{(1)}$

Step A — Evaluate the gradient at $x^{(0)}$

$$\nabla f(x^{(0)}) = \begin{bmatrix} 2(-5) \\ 4(-2) \end{bmatrix} = \begin{bmatrix} -10 \\ -8 \end{bmatrix}$$

Step B — Scale by the learning rate $\alpha = 0.1$

$$\alpha \nabla f(x^{(0)}) = 0.1 \begin{bmatrix} -10 \\ -8 \end{bmatrix} = \begin{bmatrix} -1 \\ -0.8 \end{bmatrix}$$

Step C — Subtract this from $x^{(0)}$

$$x^{(1)} = x^{(0)} - \alpha \nabla f(x^{(0)})$$

$$= \begin{bmatrix} -5 \\ -2 \end{bmatrix} - \begin{bmatrix} -1 \\ -0.8 \end{bmatrix} = \begin{bmatrix} -4 \\ -1.2 \end{bmatrix}$$

So,

$$x^{(1)} = (-4, -1.2).$$

This is already closer to the origin $(0, 0)$ than $x^{(0)} = (-5, -2)$.

Iteration 2: Computing $x^{(2)}$

Step A — Evaluate the gradient at $x^{(1)} = (-4, -1.2)$

$$\nabla f(x^{(1)}) = \begin{bmatrix} 2(-4) \\ 4(-1.2) \end{bmatrix} = \begin{bmatrix} -8 \\ -4.8 \end{bmatrix}$$

Step B — Scale by $\alpha = 0.1$

$$\alpha \nabla f(x^{(1)}) = 0.1 \begin{bmatrix} -8 \\ -4.8 \end{bmatrix} = \begin{bmatrix} -0.8 \\ -0.48 \end{bmatrix}$$

Step C — Subtract from $x^{(1)}$

$$x^{(2)} = \begin{bmatrix} -4 \\ -1.2 \end{bmatrix} - \begin{bmatrix} -0.8 \\ -0.48 \end{bmatrix} = \begin{bmatrix} -3.2 \\ -0.72 \end{bmatrix}$$

So,

$$x^{(2)} = (-3.2, -0.72).$$

Again, the point moves closer to the origin.

Iteration 3: Computing $x^{(3)}$

Step A — Evaluate the gradient

$$\nabla f(x^{(2)}) = \begin{bmatrix} 2(-3.2) \\ 4(-0.72) \end{bmatrix} = \begin{bmatrix} -6.4 \\ -2.88 \end{bmatrix}$$

Step B — Scale by α

$$\alpha \nabla f(x^{(2)}) = 0.1 \begin{bmatrix} -6.4 \\ -2.88 \end{bmatrix} = \begin{bmatrix} -0.64 \\ -0.288 \end{bmatrix}$$

Step C — Update the point

$$x^{(3)} = \begin{bmatrix} -3.2 \\ -0.72 \end{bmatrix} - \begin{bmatrix} -0.64 \\ -0.288 \end{bmatrix} = \begin{bmatrix} -2.56 \\ -0.432 \end{bmatrix}$$

Iterations 4–6 (Same Procedure, Condensed)

Iteration 4

$$\nabla f(x^{(3)}) = \begin{bmatrix} -5.12 \\ -1.728 \end{bmatrix}$$

$$\alpha \nabla f(x^{(3)}) = \begin{bmatrix} -0.512 \\ -0.1728 \end{bmatrix}$$

$$x^{(4)} = \begin{bmatrix} -2.048 \\ -0.2592 \end{bmatrix}$$

Iteration 5

$$\nabla f(x^{(4)}) = \begin{bmatrix} -4.096 \\ -1.0368 \end{bmatrix}$$

$$\alpha \nabla f(x^{(4)}) = \begin{bmatrix} -0.4096 \\ -0.10368 \end{bmatrix}$$

$$x^{(5)} = \begin{bmatrix} -1.6384 \\ -0.15552 \end{bmatrix}$$

Iteration 6

$$\nabla f(x^{(5)}) = \begin{bmatrix} -3.2768 \\ -0.62208 \end{bmatrix}$$

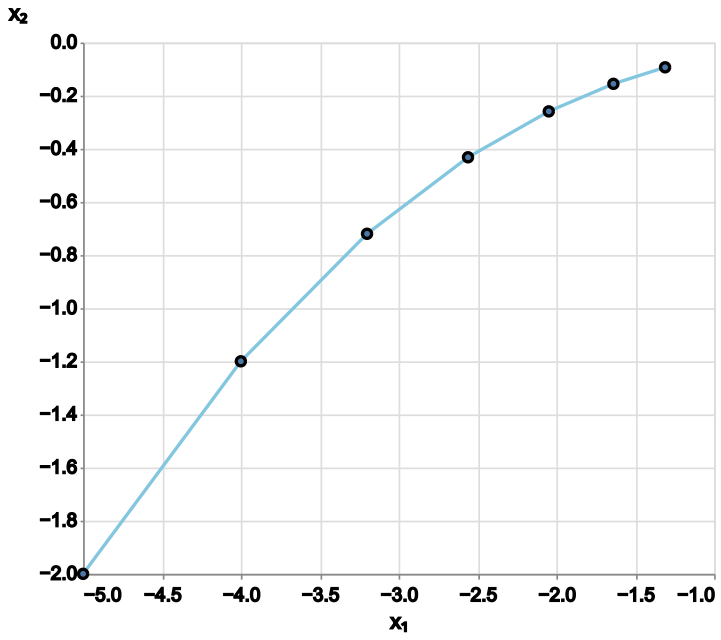
$$\alpha \nabla f(x^{(5)}) = \begin{bmatrix} -0.32768 \\ -0.062208 \end{bmatrix}$$

$$x^{(6)} = \begin{bmatrix} -1.31072 \\ -0.093312 \end{bmatrix}$$

Complete Iteration Table

Iteration (t)	$x_1^{(t)}$	$x_2^{(t)}$	$\nabla f(x^{(t)})$	Distance from Origin (Approx.)
0	-5.00000	-2.00000	$(-10, -8)$	5.39
1	-4.00000	-1.20000	$(-8, -4.8)$	4.18
2	-3.20000	-0.72000	$(-6.4, -2.88)$	3.28
3	-2.56000	-0.43200	$(-5.12, -1.728)$	2.60
4	-2.04800	-0.25920	$(-4.096, -1.0368)$	2.06
5	-1.63840	-0.15552	$(-3.2768, -0.62208)$	1.65
6	-1.31072	-0.093312	$(-2.62144, -0.373248)$	1.31

Visualization of the Optimization Path



Observation

Notice how the step size shrinks automatically over the iterations—from a gradient of $(-10, -8)$ at the start down to approximately $(-3.28, -0.62)$ by iteration 5.

This happens because, for a quadratic bowl, the gradient magnitude is proportional to the distance from the minimum:

- **Far from the minimum:** large gradient → large update steps.
- **Near the minimum:** small gradient → small update steps.

As a result, Gradient Descent naturally **decelerates** as it approaches the optimum, even when the learning rate α remains constant.

7. Gradient Ascent for Logistic Regression

The same idea applies to **maximization** problems—here, maximizing the likelihood of a binary, linearly-separable classification problem. The predictor is parameterized by θ via the **sigmoid function**, and its output is treated as a probability:

$$h_\theta(x_i) = \frac{1}{1 + e^{-\theta^\top x_i}}$$

> **Algorithm: Gradient Ascent for Logistic Regression**
>
> **Input:** Training dataset
>
> $$\{(x_1, y_1), (x_2, y_2), \ldots, (x_m, y_m)\}$$
>
> 1. Initialize $\theta^{(0)}$ (randomly, or using prior knowledge of the data) and learning rate α.
> 2. Set $t \leftarrow 0$.
> 3. **While** not converged **do**
> 4. **For** $j \in \{0, 1, 2, \ldots, d\}$ **do**
>
> $$ \theta_j^{(t+1)} \leftarrow \theta_j^{(t)}
>
> +
>
> 5. **End for**
> 6. Set $t \leftarrow t + 1$.
> 7. **End while**
>
> **Output:** Optimal classifier h_{θ^*}.

Notice the **plus sign** in the update rule. Since we are **maximizing** the likelihood (gradient ascent), we move **along** the gradient instead of against it.

The gradient here was derived previously. The important structural detail—which motivates everything in the next section—is that updating **a single parameter** θ_j requires a **summation over all** m **training examples**.

8. The Computational Bottleneck — Why We Need Something Better

Look closely at the update equation above.

To update just one parameter θ_j, you must compute a summation over **all** m **training data points**:

$$\sum_{i=1}^{m} (y_i - h_\theta(x_i))\, x_{ij}.$$

For a large dataset—say, hundreds of thousands or millions of training points—this becomes computationally expensive, and it must be repeated:

- for **every parameter** in the model, and
- at **every iteration** of gradient ascent or gradient descent.

As datasets become larger and more complex, models usually require **more parameters** to capture the underlying patterns. This increases the computational cost even further.

Example: Linear Regression Has the Same Problem

Consider linear regression, where the loss function is

$$L(\theta) = \sum_{i=1}^{n} e_i^2 = \sum_{i=1}^{n} (x_i^\top \theta - y_i)^2 .$$

Here,

- y_i is the ground-truth label.
- $x_i^\top \theta$ is the model's prediction.

Dividing by n does not change the location of the minimum, so we often write the loss in the more general form

$$L(\theta) = \frac{1}{n} \sum_{i=1}^{n} f_i(x_i, \theta).$$

Taking the gradient with respect to θ gives

$$\nabla_\theta L(\theta) = \frac{1}{n} \sum_{i=1}^{n} \nabla f_i.$$

The Universal Computational Pattern

The same **summation over the entire dataset** appears in almost every machine learning objective function.

Whether it is:

- Linear Regression
- Logistic Regression
- Neural Networks
- Support Vector Machines
- Deep Learning models

the gradient is typically computed as an aggregate over all training examples.

This is the fundamental computational bottleneck of **Batch Gradient Descent**, and it motivates faster optimization methods such as **Stochastic Gradient Descent (SGD)** and **Mini-Batch Gradient Descent**.

> 📝 **Note for Review:** This is the exact motivation for Stochastic Gradient Descent — instead of asking "how do I make gradient descent more accurate," the question becomes "how do I make each *step* of gradient descent computationally cheap enough to actually run on large data?"

9. Stochastic Gradient Descent (SGD)

Given the loss form

$$f(x) = \frac{1}{n} \sum_{i=1}^{n} f_i(x),$$

the exact Gradient Descent update is

$$x^{(t+1)} = x^{(t)} - \alpha \frac{1}{n} \sum_{i=1}^{n} \nabla_x f_i(x),$$

which is computationally expensive for large n.

The SGD Idea

Instead of summing over the **entire dataset**, randomly pick a **single data point** i and use only its gradient as an (unbiased) estimate of the true gradient:

$$x^{(t+1)} = x^{(t)} - \alpha \nabla_x f_i(x).$$

This drastically simplifies each update—no summation over the whole dataset is needed.

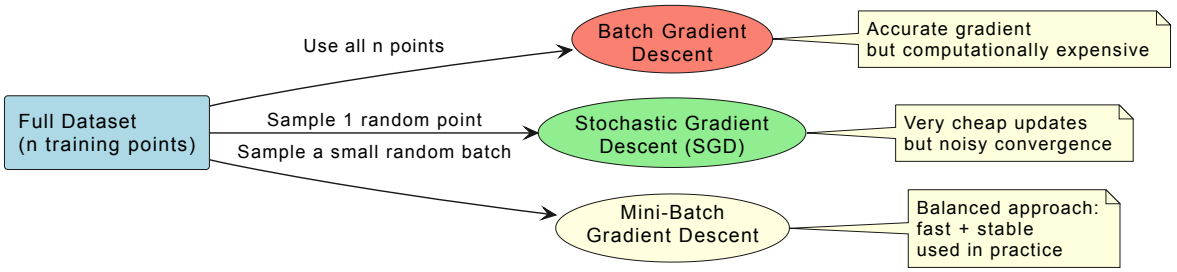
It can be shown (proof omitted here, available in standard references) that this estimate is **sufficient for convergence**, although the optimization trajectory will show **fluctuations** rather than a perfectly smooth path.

In Practice: Mini-Batch SGD

Rather than using a single data point, a **small random batch (mini-batch)** is used to approximate the gradient. This is called **Mini-Batch SGD**.

It retains the computational benefit while also making efficient use of GPU/CPU parallelism (since batches vectorize well), and the mini-batch estimate is less noisy than using a single point.

Batch Gradient Descent vs SGD vs Mini-Batch SGD



Behavior

Using the same function and starting point as the earlier example, Mini-Batch SGD (or single-point SGD) follows a path similar to full-batch Gradient Descent.

However, as it approaches the minimum, it begins to **fluctuate** around the optimal solution instead of settling smoothly.

This happens because a single data point (or a small batch) provides only a **noisy approximation** of the true gradient, and this noise does not disappear completely near the optimum.

Two Ways to Reduce the Fluctuation

1. **Increase the batch size.**

   A larger mini-batch reduces the variance (noise) of the gradient estimate, producing a smoother optimization path.

2. **Use a dynamically decaying learning rate.**

   As training progresses, gradually reduce the step size so that noisy updates become smaller near the optimum.

10. The Dynamic Learning Rate Trick

Instead of keeping α fixed for every iteration, make it a function of the iteration number t:

$$\alpha(t) = \alpha_0 e^{-\lambda t},$$

where

- α_0 is the initial learning rate.
- λ is the decay rate.
- t is the iteration number.

Why This Works

- **Early training (small t):** The learning rate is close to α_0, so updates are large and optimization progresses quickly.
- **Later training (large t):** The exponential factor shrinks toward zero, making updates progressively smaller.

As the learning rate decreases, the fluctuations caused by noisy SGD gradients become much smaller, allowing the optimizer to settle near the optimum.

Exponential Learning Rate Decay

$$\boxed{\alpha(t) = \alpha_0 e^{-\lambda t}}$$

This is called **exponential learning rate decay**.

> **Note for Review:** This decaying-learning-rate idea reappears (in different clothing) inside **AdaGrad**, **RMSProp**, and **Adam**. All of them effectively reduce the **effective learning rate** during training, although they use gradient statistics (rather than just the iteration number) to determine how much the step size should shrink.---

11. The Dimensional Scaling Problem

When One Learning Rate Isn't Enough

So far, every example used a function where gradients in different directions were roughly comparable in scale. But consider

$$f(x_1, x_2) = 0.1x_1^2 + 2x_2^2.$$

Gradient

$$\nabla f(x_1, x_2) = \begin{bmatrix} 0.2x_1 \\ 4x_2 \end{bmatrix}.$$

At the point $(10, 10)$,

$$\nabla f = (0.2 \times 10,\ 4 \times 10) = (2,\ 40).$$

Multiplying by a learning rate α:

- Step in the x_1 direction:

$$2\alpha$$

- Step in the x_2 direction:

$$40\alpha$$

This is a **20× difference in step size** for the same learning rate.

If

$$\alpha = 0.1,$$

then

- x_1 step = **0.2**
- x_2 step = **4**

This means **one direction has a very gently sloping surface** (small gradient, slow convergence), while **another direction has a very steeply sloping surface** (large gradient, risk of overshoot or divergence). A *single, shared* learning rate cannot be ideal for both simultaneously.

Experiment 1 — $\alpha = 0.4$

Starting from $(-5, -2)$:

- The x_2 **direction** (steep) converges quickly, reaching close to $x_2 = 0$ in about **5–7 iterations**, because its gradient is large and $\alpha = 0.4$ is still small enough to avoid divergence.
- The x_1 **direction** (gentle) is painfully slow. After the same 7 iterations, x_1 has only moved from **−5** to around **−2.5**, because the gradient along x_1 is small and becomes even smaller as the point approaches the optimum.

Given enough iterations, this still converges—but very slowly along the x_1 direction.

Experiment 2 — $\alpha = 0.6$

Now the x_1 **direction** moves nicely from **−5** toward **0**.

However, the x_2 **direction diverges**.

Because the gradient in the steep direction is much larger, multiplying it by the larger learning rate causes the update to overshoot the minimum by an increasing amount at every iteration.

By around **iteration 15**, x_2 has exploded from **−2** to roughly **1000**, oscillating and diverging further with additional iterations.

The Core Dilemma

> *You cannot pick a single learning rate that is simultaneously safe for a steep direction and fast for a gentle direction.*

This is precisely the problem that motivates **Momentum** and adaptive learning-rate methods such as **AdaGrad**, **RMSProp**, and **Adam**.

---

12. Momentum — "Don't Lose the Momentum"

Looking back at the plain update rule, notice that to go from one point to the next, we only used the gradient computed at the **current point**. The gradients from earlier steps were discarded.

The Momentum Idea

Instead of using only the most recent gradient, **aggregate gradients from previous steps as well**.

To update from the current point, use information from gradients computed at previous iterations, with different weights controlled by a parameter β.

In other words, **don't lose the momentum you've already built up.**

Update Equations

The momentum update is

$$v_t = \beta v_{t-1} + g_{t,t-1},$$

where

$$g_{t,t-1} = \nabla_x \left(\frac{1}{|\mathrm{B}_t|} \sum_{i \in \mathrm{B}_t} f_i(x_{t-1}) \right).$$

The parameter update becomes

$$x_t = x_{t-1} - \alpha_t v_t.$$

Here,

- $g_{t,t-1}$ is the gradient computed on the current mini-batch (or full batch).
- v_t is the running weighted aggregate, often called the **velocity**.
- β controls how much past gradients are remembered.

Why Momentum Fixes Both Problems

Gentle Direction (e.g. x_1)

The gradient is small but consistently points in the **same direction** at every iteration.

Adding together many small, aligned gradients produces a **larger velocity**, increasing the effective step size and accelerating convergence along the gentle direction.

Steep / Oscillating Direction (e.g. x_2)

The gradient repeatedly flips sign as the optimization overshoots the minimum.

For example,

- positive,
- negative,
- positive,
- negative.

When these gradients are accumulated, positive and negative contributions **partially cancel each other**, reducing the effective step size and damping oscillations.

The Big Picture

Gradient aggregation helps in **both** failure modes simultaneously:

- **Amplifies** small, consistent gradients in shallow directions.
- **Cancels** large, oscillating gradients in steep directions.

As a result, momentum produces a smoother optimization trajectory and typically converges faster than plain Gradient Descent.

With momentum (for example, $\beta = 0.25$), the earlier slow-convergence/divergence problem is greatly reduced, and the trajectory converges smoothly along both axes.

The gradient used in $g_{t,t-1}$ may come from:

- Full Batch Gradient Descent,
- Stochastic Gradient Descent (SGD), or
- Mini-Batch SGD.

Momentum works with all of these optimization frameworks.

> **Note for Review:** Don't set β too aggressively. Since momentum accumulates gradients over many iterations, an overly large β can initially create excessively large updates in some directions. Although momentum is designed to stabilize optimization, β itself is an important hyperparameter that must be tuned.

13. AdaGrad — Adaptive Gradient

Momentum aggregates gradients to build velocity, but keeps a **single, shared** learning rate across all dimensions. **AdaGrad** instead gives **each parameter its own adaptive learning rate**, based on how large that parameter's historical gradients have been—directly targeting the problem from Section 11.

Here the notation switches to a machine learning framing:

- x_t is the t-th training data point.
- ω represents the model's weights (parameters).

Update Equations

First compute the gradient:

$$g_t = \nabla_\omega \ell(y_t, f(x_t, \omega)) .$$

Accumulate the squared gradients:

$$s_t = s_{t-1} + g_t^2 .$$

Update the parameters:

$$\omega_t = \omega_{t-1} - \frac{\alpha}{\sqrt{s_t + \epsilon}} g_t$$

What Each Term Means

- g_t is the **gradient vector** (one component per parameter or feature).
- s_t is a **vector** that accumulates the **element-wise square** of the gradient over time.
- g_t^2 means **square each component individually** (not a dot product).
- Squaring, square roots, and division are all **element-wise operations**.
- ϵ is a small safety constant that prevents division by zero when a parameter has received zero gradient updates.

Why AdaGrad Helps

Unlike ordinary Gradient Descent, where the same learning rate α multiplies every parameter update, AdaGrad gives each parameter its own **effective learning rate**:

$$\frac{\alpha}{\sqrt{s_t + \epsilon}},$$

This quantity is a **vector**, so every parameter gets a different step size.

- **Large historical gradients:** s_t becomes large, so the effective learning rate **shrinks**.
- **Small or infrequent gradients:** s_t stays small, so the effective learning rate remains **relatively large**.

This is especially useful for machine learning datasets containing:

- **Frequent features:** receive many large gradient updates.
- **Rare features:** receive sparse or small gradient updates.

AdaGrad automatically slows learning for frequent features while continuing to learn quickly for rare ones.

> **Note for Review — The Flaw in AdaGrad:** s_t is a cumulative **running sum** of squared gradients. It only grows over time—it never decreases.

Therefore,

$$\frac{\alpha}{\sqrt{s_t + \epsilon}}$$

monotonically shrinks toward zero, even if the model has not converged.

In practice, AdaGrad often becomes so conservative after many iterations that training nearly stops.

This exact weakness motivates **RMSProp**.

14. RMSProp

RMSProp (**Root Mean Square Propagation**) fixes AdaGrad's "ever-growing denominator" problem.

Instead of storing a cumulative sum, it stores an **exponentially weighted moving average** of squared gradients, controlled by a decay parameter γ.

Update Equations

Compute the gradient:

$$g_t = \nabla_\omega \ell(y_t, f(x_t, \omega)) .$$

Update the running average:

$$s_t = \gamma s_{t-1} + (1-\gamma) g_t^2 .$$

Update the parameters:

$$\omega_t = \omega_{t-1} - \frac{\alpha}{\sqrt{s_t + \epsilon}} g_t$$

Comparing AdaGrad and RMSProp

AdaGrad uses

$$s_t = s_{t-1} + g_t^2 ,$$

so every squared gradient is remembered forever.

RMSProp uses

$$s_t = \gamma s_{t-1} + (1-\gamma) g_t^2 ,$$

which gives exponentially less weight to older gradients.

The decay parameter γ controls the trade-off:

- Larger $\gamma \rightarrow$ longer memory.
- Smaller $\gamma \rightarrow$ recent gradients matter more.

Why RMSProp Works Better

Because old gradient information gradually decays,

- s_t remains bounded.
- s_t can even decrease if recent gradients become smaller.
- The effective learning rate no longer collapses permanently toward zero.

As a result, RMSProp usually continues learning much longer than AdaGrad.

15. Adam — Adaptive Moment Estimation

Adam combines **Momentum** with **RMSProp**.

It keeps track of:

1. A moving average of gradients (**first moment**).
2. A moving average of squared gradients (**second moment**).

Update Equations

Step 1 — Compute the Gradient

$$g_t = \nabla_\omega \ell(y_t, f(x_t, \omega)) .$$

Step 2 — First Moment (Momentum)

$$v_t = \beta_1 v_{t-1} + (1 - \beta_1) g_t.$$

Step 3 — Second Moment (RMSProp)

$$s_t = \beta_2 s_{t-1} + (1 - \beta_2) g_t^2.$$

Step 4 — Bias Correction

$$\hat{v}_t = \frac{v_t}{1 - \beta_1^t},$$

$$\hat{s}_t = \frac{s_t}{1 - \beta_2^t}.$$

Step 5 — Parameter Update

$$\omega_t = \omega_{t-1} - \frac{\alpha \hat{v}_t}{\sqrt{\hat{s}_t + \epsilon}}.$$

How the Pieces Fit Together

First Moment (v_t)

The first moment is the **momentum-style moving average** of the raw gradient.

Because gradients keep their signs,

- positive gradients,
- negative gradients,

can partially cancel each other in oscillating directions.

This produces a smoother, more stable update direction.

Second Moment (s_t)

The second moment is the **RMSProp-style moving average** of squared gradients.

Since squared gradients are always non-negative,

- they never cancel,
- they track the typical gradient magnitude for each parameter.

This determines the adaptive learning rate for every parameter individually.

Bias Correction

Early in training, both v_t and s_t start near zero.

Without correction, the moving averages are biased toward zero during the first few iterations.

Adam corrects this by dividing by

$$1-\beta_1^t \quad \text{and} \quad 1-\beta_2^t,$$

so early updates are not artificially suppressed.

Final Interpretation

The final Adam update combines both mechanisms:

- **Momentum ($\hat{v}_t$):** determines the update direction and smooths oscillations.
- **Adaptive scaling ($\hat{s}_t$):** adjusts the learning rate independently for every parameter.

In short,

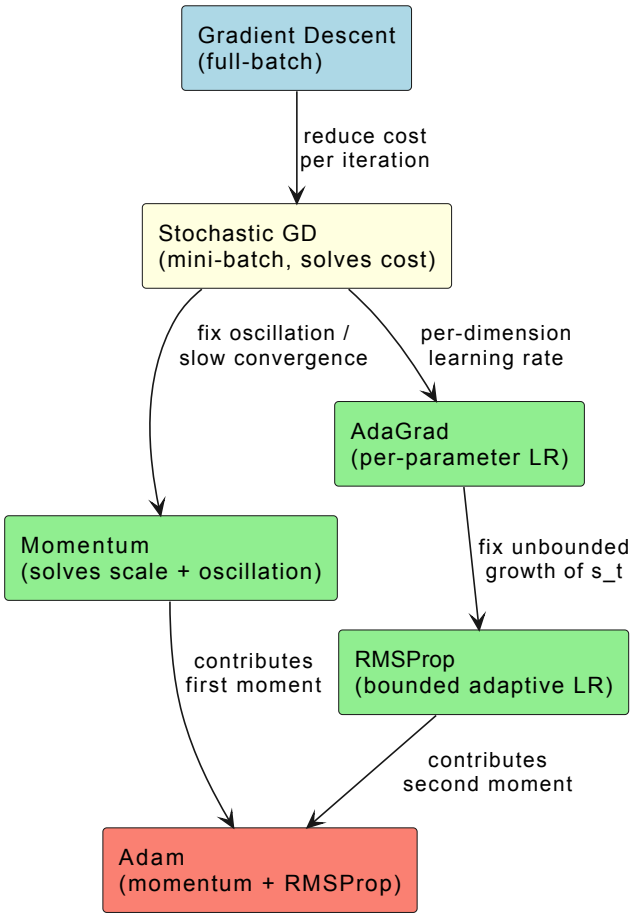
$$\boxed{\text{Adam} = \text{Momentum} + \text{RMSProp}}$$

or, conceptually:

> **Adam = Momentum's "which direction and how big a step" + RMSProp's "how much to trust and scale that step for each parameter."**

16. Summary — Comparing All the Variants

Algorithm	Core Idea	Fixes	Key Formula Component
Gradient Descent	Move against the full-dataset gradient.	Baseline optimality guarantee (via Taylor series).	x_{t+1} = x_t - \alpha \nabla f(x_t)
SGD (mini-batch)	Approximate the gradient using 1 point / a small batch.	Computational cost of summing over the whole dataset.	x_{t+1} = x_t - \alpha \nabla f_i(x_t)
Dynamic LR	Decay \alpha over iterations.	Fluctuation near the optimum caused by noisy SGD gradients.	\alpha(t) = \alpha_0 e^{-\lambda t}
Momentum	Aggregate gradients across past steps.	Slow convergence (gentle direction) *and* oscillation/divergence (steep direction), simultaneously.	v_t = \beta v_{t-1} + g_t
AdaGrad	Give each parameter its own learning rate, shrinking with its historical squared-gradient sum.	A single shared \alpha being wrong for different-scale directions.	s_t = s_{t-1} + g_t^2
RMSProp	Same as AdaGrad, but use an exponential moving average instead of an ever-growing sum.	AdaGrad's learning rate collapsing to zero permanently.	s_t = \gamma s_{t-1} + (1-\gamma)g_t^2
Adam	Combine Momentum's signed gradient aggregate with RMSProp's adaptive scaling, plus bias correction.	Needing both directionally-smart *and* magnitude-aware updates at once.	\omega_t = \omega_{t-1} - \dfrac{\alpha \hat v_t}{\sqrt{\hat s_t}+\epsilon}



17. Practice Questions

Think through these rather than just recalling formulas—they're designed to test whether the *intuition* behind each method has actually landed.

1. **One Learning Rate vs Anisotropic Loss Surfaces**

   Suppose you run plain Gradient Descent (no momentum, no adaptive learning rate) on a strongly elongated elliptical bowl, much like $f(x_1, x_2) = 0.1x_1^2 + 2x_2^2$, and you are told you are **only allowed one fixed learning rate** for the entire run.

   Explain why no single choice of α can give you both **fast convergence along** x_1 and **guaranteed non-divergence along** x_2 at the same time. What does this imply about the shape (the **condition number**) of a loss surface, and how directly it affects optimizer choice?

2. **Taylor Series and Curvature**

   In the Taylor-series proof of Gradient Descent (Section 4), the guarantee $f(x) > f(x - \eta\nabla f)$ relied on dropping the second-order (Hessian) term.

   Construct an argument for why, near a very sharp or narrow minimum (large curvature, i.e., large Hessian eigenvalues), even a learning rate that works perfectly well near a wide, shallow minimum could cause divergence. What does this tell you about why a **one-size-fits-all learning rate** is fundamentally fragile?

3. **Why RMSProp Doesn't Just Reset AdaGrad**

   AdaGrad's accumulator s_t can only grow, never shrink, causing learning to effectively halt over long training runs.

   Suppose instead of switching to RMSProp's exponential moving average, someone proposes simply resetting $s_t = 0$ every K iterations.

   Reason through what problems this crude fix would create that RMSProp's smooth decay avoids. Think about what happens immediately **before** a reset and immediately **after** one.

4. **Why Adam Needs Raw Gradients in the First Moment**

   In Adam, the first-moment term v_t is a signed running average of the **raw gradient**, while the second-moment term s_t uses the **squared gradient**.

   Explain why Adam's design would break if **both** moments used squared gradients (i.e., if v_t also accumulated g_t^2 instead of g_t). Specifically, what critical piece of directional information would be lost, and which earlier algorithm's key benefit would disappear as a result?

5. **Why Bias Correction Exists**

   Bias correction in Adam is $\hat{v}_t = \frac{v_t}{1-\beta_1^t}$. It matters most when t is small and gradually becomes negligible because $\beta_1^t \to 0$.

   Suppose you initialize $\beta_1 = 0.9$ and **skip bias correction entirely**. Work through what the **very first update** ($t = 1$) looks like when $v_0 = 0$.

   Explain concretely how the uncorrected first step differs from the true gradient direction and magnitude, why the first update is much smaller than intended, and why this is primarily a **cold-start problem** that fades away as training progresses rather than a persistent flaw.

Support Vector Machines — Geometric Margins & The Maximum-Margin Classifier

Machine Learning 2 — Lecture 5 (Indian Institute of Technology Jodhpur)

Index

- 1. Where We Left Off: From Optimization Theory to Learning Algorithms — *the bridge between "math class" and "actual machine learning."*
- 2. The Geometric Margin: How Far Is a Point From a Line? — *some vector-algebra gymnastics to measure distance without a ruler.*
- 3. Scale Invariance: The Hyperplane's Best-Kept Secret — *why multiplying your parameters by a random positive number changes nothing.*
- 4. Setting Up the Support Vector Machine — *SVM's peculiar taste for* ± 1 *instead of* $0/1$.
- 5. Why Maximize the Margin? A Tale of Infinite Classifiers — *when "any line that separates the dots" just isn't good enough.*
- 6. Turning Intuition Into Constraints — *how "positive side / negative side" becomes one tidy inequality.*
- 7. The (Messy) Maximum-Margin Optimization Problem — *a max-min sandwich nobody asked for.*
- 8. The Scaling Trick: Cleaning Up the Sandwich — *SVM's favorite party trick, performed once again.*
- 9. Support Vectors: The VIPs of the Dataset — *the only points that actually matter.*
- 10. When Classes Refuse to Separate: Slack Variables — *SVM learns to forgive (a little).*
- 11. Rewriting Everything as a Single Matrix Problem — *turning* n *separate constraints into one glorious matrix inequality.*
- 12. A Warm-Up: Solving an Equality-Constrained Problem — *eigenvectors make a surprise guest appearance.*
- 13. Think-It-Through Questions — *no googling allowed.*

1. Where We Left Off: From Optimization Theory to Learning Algorithms

Until this point in the course, the focus has been squarely on the *machinery* of optimization — gradients, constrained optimization, and the algorithms used to solve them. Today marks a turning point: this machinery is finally put to work on an actual machine learning technique. The first learning algorithm to benefit from this optimization toolkit is the **Support Vector Machine (SVM)**.

SVM is fundamentally a **binary classifier**, although with the right modifications it can be extended to handle multi-class classification as well. For this lecture, the discussion is restricted to the case where the **two classes are linearly separable** — meaning a straight line (or, in higher dimensions, a flat hyperplane) can perfectly divide the two classes.

Before SVM itself can be defined, however, a purely geometric question needs to be answered: *given a hyperplane and a data point, how far is that point from the hyperplane?* This quantity — the **margin** — turns out to be the single most important concept in the entire derivation.

2. The Geometric Margin: How Far Is a Point From a Line?

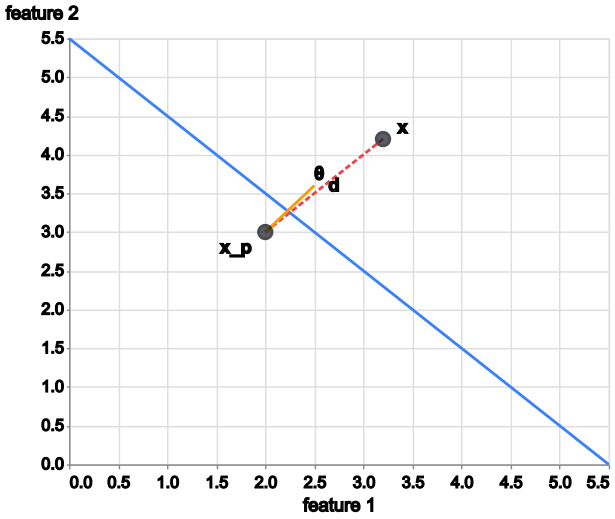
Consider a hyperplane

$$\mathrm{H} = \{\mathbf{x} : \boldsymbol{\theta}^\top \mathbf{x} + b = 0\}$$

A line (or hyperplane, in general) can be parameterized in many ways, but here it is described by two things:

- $\boldsymbol{\theta}$ — the **normal vector**, perpendicular to the hyperplane.
- b — related to the **distance of the hyperplane from the origin**.

Now take an arbitrary data point $\mathbf{x}$ that does *not* lie on the hyperplane, and let $\mathbf{x}_p$ be the point **on** the hyperplane that is closest to $\mathbf{x}$ (i.e., the foot of the perpendicular from $\mathbf{x}$). The vector $\mathbf{d}$ joining $\mathbf{x}_p$ to $\mathbf{x}$ represents exactly the distance we want to compute.



Deriving the distance step by step.

By definition, the vector $\mathbf{d}$ points in the same direction as the normal vector $\boldsymbol{\theta}$ (since $\mathbf{d}$ is perpendicular to the hyperplane, and $\boldsymbol{\theta}$ is also perpendicular to it by construction). It may, however, have a different magnitude. So we can write:

$$\mathbf{x} - \mathbf{x}_p = \alpha\, \boldsymbol{\theta}$$

for some unknown scalar α. Since $\mathbf{x}_p$ lies *on* the hyperplane, it must satisfy the hyperplane equation:

$$\boldsymbol{\theta}^\top \mathbf{x}_p + b = 0$$

Substituting $\mathbf{x}_p = \mathbf{x} - \alpha\boldsymbol{\theta}$ into this equation:

$$\boldsymbol{\theta}^\top (\mathbf{x} - \alpha\boldsymbol{\theta}) + b = 0$$

Expanding and solving for α:

$$\alpha = \frac{\boldsymbol{\theta}^\top \mathbf{x} + b}{\boldsymbol{\theta}^\top \boldsymbol{\theta}}$$

Now, the distance itself is just the magnitude of $\mathbf{d}$:

$$\|\mathbf{d}\|_2 = \|\mathbf{x} - \mathbf{x}_p\|_2 = \|\alpha\boldsymbol{\theta}\|_2$$

Substituting the value of α found above, and using the fact that $\boldsymbol{\theta}^\top\boldsymbol{\theta} = \|\boldsymbol{\theta}\|_2^2$ (so one factor of $\|\boldsymbol{\theta}\|_2$ cancels):

$$\|\mathbf{d}\|_2 = \frac{|\boldsymbol{\theta}^\top \mathbf{x} + b|}{\|\boldsymbol{\theta}\|_2}$$

This is the **geometric margin** of a point $\mathbf{x}$ with respect to a hyperplane $(\boldsymbol{\theta}, b)$ — a clean, closed-form expression for "how far is this point from my classifier."

> **Note for Review:** Students often forget *why* the denominator is $\|\boldsymbol{\theta}\|_2$ and not $\|\boldsymbol{\theta}\|_2^2$. It comes purely from $\|\alpha\boldsymbol{\theta}\|_2 = |\alpha|\,\|\boldsymbol{\theta}\|_2$, and α itself already carries one power of $\|\boldsymbol{\theta}\|_2^{-2}$ in its denominator — one power cancels against the $\|\boldsymbol{\theta}\|_2$ picked up from taking the norm, leaving exactly one power of $\|\boldsymbol{\theta}\|_2$ in the final denominator.

3. Scale Invariance: The Hyperplane's Best-Kept Secret

Here's a subtle but extremely important property. Suppose the parameters $(\boldsymbol{\theta}, b)$ representing a hyperplane are scaled by some **positive constant** γ, giving a new pair $(\gamma\boldsymbol{\theta}, \gamma b)$. Does the hyperplane change?

$$\gamma\boldsymbol{\theta}^\top \mathbf{x} + \gamma b = 0 \quad \Longrightarrow \quad \boldsymbol{\theta}^\top \mathbf{x} + b = 0$$

Since $\gamma \neq 0$, it can simply be divided out — the equation reduces back to the original hyperplane. **The scaled parameters represent exactly the same hyperplane.**

Now the natural follow-up: does the *margin* change under this scaling?

$$\frac{|\gamma\boldsymbol{\theta}^\top \mathbf{x} + \gamma b|}{\|\gamma\boldsymbol{\theta}\|_2} = \frac{\gamma\,|\boldsymbol{\theta}^\top \mathbf{x} + b|}{\gamma\,\|\boldsymbol{\theta}\|_2} = \frac{|\boldsymbol{\theta}^\top \mathbf{x} + b|}{\|\boldsymbol{\theta}\|_2}$$

Both the numerator and denominator pick up the same factor of γ (since $\gamma > 0$), which cancels out completely.

Conclusion: Both the hyperplane *and* its geometric margin are invariant to positive scaling of $(\boldsymbol{\theta}, b)$. This is a trivial fact to prove, but it will soon become the single most useful trick in solving the SVM optimization problem — because it means there is a whole family of $(\boldsymbol{\theta}, b)$ pairs that all represent the same classifier, and among that family, one can always pick the most *convenient* representative.

4. Setting Up the Support Vector Machine

With the geometric toolkit ready, the Support Vector Machine itself can now be defined.

Consider a labeled dataset:

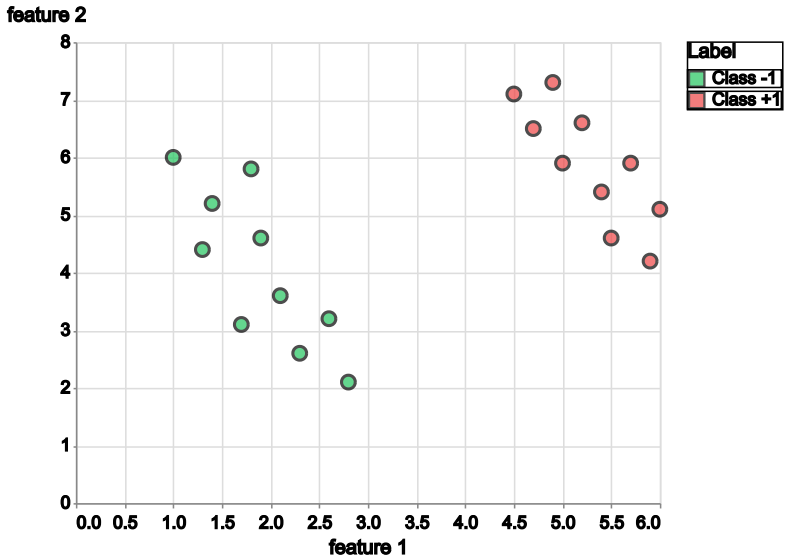
$$\mathrm{D} = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \ldots, (\mathbf{x}_n, y_n)\}$$

where each data point $\mathbf{x}_i$ has a corresponding binary label y_i. Here comes SVM's one quirky convention: instead of using $\{0, 1\}$ as in logistic regression, SVM labels the two classes as

$$y_i \in \{-1, +1\}$$

There is nothing deep about this choice — it is simply more convenient algebraically, as will become clear very soon when the two "positive side" and "negative side" conditions get merged into a single inequality.

The goal is to find a **linear boundary** — a hyperplane — that separates the two classes.



Since the two classes above are linearly separable, a classifier can indeed be found — but, as the next section shows, *which* classifier to pick is far from obvious.

5. Why Maximize the Margin? A Tale of Infinite Classifiers

Looking at the scatter plot above, it's tempting to think the job is done once *any* line correctly separates the green and red points. But here's the catch: there are **infinitely many** such lines. Any small rotation or shift of a valid separating line, as long as it doesn't cross into the other class's territory, is still a perfectly valid classifier by the "correctly separates the data" criterion alone.

So which one should be chosen? Consider two extreme choices:

- A classifier that **hugs the red class** very closely.
- A classifier that **hugs the green class** very closely.

In both cases, a small amount of measurement noise could push a borderline point across the classifier, causing a misclassification. A point sitting very close to the boundary is *fragile* — the slightest perturbation flips its predicted class.

The fix is intuitive: choose the classifier that stays **as far away as possible from both classes simultaneously** — i.e., the one with the **maximum margin**. This gives the classifier the largest possible "buffer zone" to absorb noise before any point gets misclassified.

This reframes the classification problem: instead of *"find a linear boundary that separates the two classes,"* the goal becomes *"find a linear boundary that separates the two classes **and** maximizes the margin between them."*

Since the margin of an individual point is just its distance from the classifier, and every dataset has many points at varying distances, the natural next question is: *the margin of **which** point?* Points that are already far from the boundary are of no real concern — they will not flip sides easily. The points that matter are the ones **closest** to the boundary on each side. Hence, the objective becomes: **maximize the minimum margin** across the entire dataset.

---

6. Turning Intuition Into Constraints

Before margins can be maximized, the "correctly separates the two classes" requirement needs to be written as a precise mathematical constraint.

Consider the classifier $\mathbf{x}^\top \boldsymbol{\theta} + b = 0$, with normal vector $\boldsymbol{\theta}$ and distance-from-origin parameter b. Take a data point $\mathbf{x}_i$ lying on the **positive side** of this hyperplane (the side $\boldsymbol{\theta}$ points toward). Since $\mathbf{x}_i$ is not on the line itself:

$$\mathbf{x}_i^\top \boldsymbol{\theta} + b \overset{\text{def}}{=} 0, \quad \text{and specifically} \quad \mathbf{x}_i^\top \boldsymbol{\theta} + b > 0$$

All points on this side are, by convention, labeled $y_i = +1$.

Now take a point $\mathbf{x}_j$ on the **negative side**:

$$\mathbf{x}_j^\top \boldsymbol{\theta} + b < 0, \quad \text{labeled } y_j = -1$$

Here is where the $\{-1, +1\}$ labeling convention pays off. Multiply the positive-side inequality by $y_i = +1$: the inequality direction is unchanged, and it stays positive. Multiply the negative-side inequality by $y_j = -1$: the inequality **flips**, turning a negative quantity into a positive one:

$$y_j \, (\mathbf{x}_j^\top \boldsymbol{\theta} + b) > 0$$

Both cases — positive-side and negative-side — now collapse into the exact same single condition:

$$y_i \, (\boldsymbol{\theta}^\top \mathbf{x}_i + b) > 0, \quad \text{for all } i = 1, \ldots, n$$

This one inequality, applied to every training point, guarantees that all points are on the *correct* side of the classifier. (For now, the possibility of a point sitting exactly on the boundary — margin equal to zero — is set aside, which is why a strict > 0 boundary rather than ≥ 0 will be used going forward.)

7. The (Messy) Maximum-Margin Optimization Problem

Combining everything so far: the goal is to find $(\boldsymbol{\theta}, b)$ that (a) correctly separates the two classes, and (b) maximizes the *minimum* margin over all data points. This is written as:

$$\gamma(\boldsymbol{\theta}, b) = \min_{\mathbf{x}_i \in \mathrm{D}} \frac{|\boldsymbol{\theta}^\top \mathbf{x}_i + b|}{\|\boldsymbol{\theta}\|_2}$$

$$\max_{\boldsymbol{\theta}, b} \quad \gamma(\boldsymbol{\theta}, b) \qquad \text{such that} \qquad y_i(\boldsymbol{\theta}^\top \mathbf{x}_i + b) \geq 0$$

This is unlike anything solved earlier in the optimization module. Previously, every problem was a simple maximization or minimization of a cost function, possibly with equality constraints. This problem, on the other hand, is:

1. A genuine **constrained optimization problem** (an inequality constraint must hold for every data point), *and*
2. Nested — it's a **max of a min**, since the margin itself is defined as a minimization over data points.

Solving a max-min problem directly is unpleasant. The next section shows how the scale-invariance property from Section 3 elegantly collapses this nested structure into a much simpler single-level problem.

8. The Scaling Trick: Cleaning Up the Sandwich

Step 1 — Pull the denominator out of the minimization.

Look carefully at the margin expression:

$$\gamma(\boldsymbol{\theta}, b) = \frac{1}{\|\boldsymbol{\theta}\|_2} \min_{\mathbf{x}_i \in \mathrm{D}} |\boldsymbol{\theta}^\top \mathbf{x}_i + b|$$

The inner minimization runs over data points $\mathbf{x}_i$, while $\|\boldsymbol{\theta}\|_2$ does not depend on $\mathbf{x}_i$ at all — it can be factored completely outside the minimization. This gives:

$$\max_{\boldsymbol{\theta}, b} \quad \frac{1}{\|\boldsymbol{\theta}\|_2} \min_{\mathbf{x}_i \in \mathrm{D}} |\boldsymbol{\theta}^\top \mathbf{x}_i + b| \qquad \text{s.t.} \qquad y_i(\boldsymbol{\theta}^\top \mathbf{x}_i + b) \geq 0, \quad i \in \{1, \ldots, n\}$$

Step 2 — Use scale invariance to fix the inner minimum to exactly 1.

Recall from Section 3: scaling $(\boldsymbol{\theta}, b)$ by any positive constant changes **neither** the hyperplane **nor** the margin. This means that for *any* candidate hyperplane $(\boldsymbol{\theta}, b)$ for which $\min_i |\boldsymbol{\theta}^\top \mathbf{x}_i + b| = c$ (some positive number), a rescaled version can always be constructed:

$$\hat{\boldsymbol{\theta}} = \frac{\boldsymbol{\theta}}{c}, \qquad \hat{b} = \frac{b}{c}$$

so that:

$$\min_{\mathbf{x}_i} |\hat{\boldsymbol{\theta}}^\top \mathbf{x}_i + \hat{b}| = 1$$

Since this rescaling changes neither the hyperplane nor the margin, **there is no loss of generality** in restricting attention only to hyperplanes for which the closest point has $|\boldsymbol{\theta}^\top \mathbf{x}_i + b| = 1$. This constraint can simply be *added* to the optimization problem, and because it fixes the minimum to exactly 1, the entire inner minimization term drops out of the objective (it's just the constant 1 now):

$$\max_{\boldsymbol{\theta}, b} \quad \frac{1}{\|\boldsymbol{\theta}\|_2}$$

$$\text{s.t.} \qquad y_i(\boldsymbol{\theta}^\top \mathbf{x}_i + b) \geq 0, \quad i \in \{1, \ldots, n\}$$

$$\min_i |\boldsymbol{\theta}^\top \mathbf{x}_i + b| = 1, \quad i \in \{1, \ldots, n\}$$

Step 3 — Merge the two constraints into one.

Because $y_i(\boldsymbol{\theta}^\top \mathbf{x}_i + b) \geq 0$ for every point, the same absolute-value trick from Section 6 applies again: $|\boldsymbol{\theta}^\top \mathbf{x}_i + b| = y_i(\boldsymbol{\theta}^\top \mathbf{x}_i + b)$. So the condition $\min_i |\boldsymbol{\theta}^\top \mathbf{x}_i + b| = 1$ becomes:

$$\min_i \; y_i(\boldsymbol{\theta}^\top \mathbf{x}_i + b) = 1 \quad \Longrightarrow \quad \forall i, \;\; y_i(\boldsymbol{\theta}^\top \mathbf{x}_i + b) \geq 1$$

Note something important here: this new constraint, $y_i(\boldsymbol{\theta}^\top \mathbf{x}_i + b) \geq 1$, is *strictly stronger* than the earlier one, $y_i(\boldsymbol{\theta}^\top \mathbf{x}_i + b) \geq 0$. Any $(\boldsymbol{\theta}, b)$ satisfying the former automatically satisfies the latter — so the original ≥ 0 constraint becomes redundant and can be dropped entirely.

Step 4 — Flip maximization into minimization.

Maximizing $\dfrac{1}{\|\boldsymbol{\theta}\|_2}$ is mathematically equivalent to minimizing $\|\boldsymbol{\theta}\|_2$, which is in turn equivalent to minimizing $\boldsymbol{\theta}^\top \boldsymbol{\theta}$ (squaring is a monotonic transformation for non-negative quantities, and it conveniently removes the square root hidden inside the norm, making the problem smooth and quadratic).

This finally yields the clean, canonical **Support Vector Machine optimization problem**:

$$\min_{\boldsymbol{\theta}, b} \;\; \boldsymbol{\theta}^\top \boldsymbol{\theta}$$

$$\text{such that} \qquad y_i(\boldsymbol{\theta}^\top \mathbf{x}_i + b) \geq 1, \quad i \in \{1, \ldots, n\}$$

> **Note for Review:** This scaling trick is the crux of the entire derivation and a very common exam question. The key idea to remember: **maximizing margin subject to a max-min structure is intractable directly, but scale-invariance lets you "fix" the hardest constraint to a constant (1) without losing generality**, collapsing the problem into an ordinary single-level constrained quadratic minimization.

---

9. Support Vectors: The VIPs of the Dataset

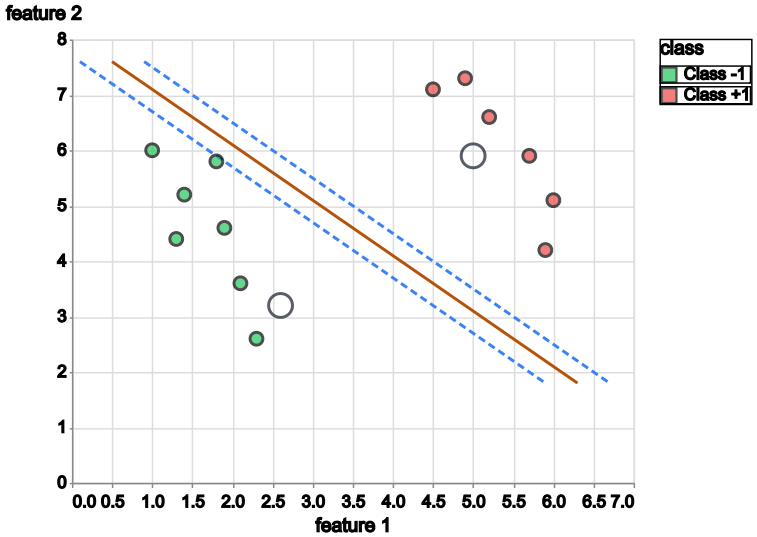
For the *optimal* parameters $(\boldsymbol{\theta}, b)$ found by solving the problem above, the two lines

$$\mathbf{x}^\top \boldsymbol{\theta} + b = 1 \qquad \text{and} \qquad \mathbf{x}^\top \boldsymbol{\theta} + b = -1$$

are not arbitrary — they pass exactly through the nearest data points of the two classes. These are precisely the data points for which the constraint is **tight**, i.e.,

$$y_i(\boldsymbol{\theta}^\top \mathbf{x}_i + b) = 1$$

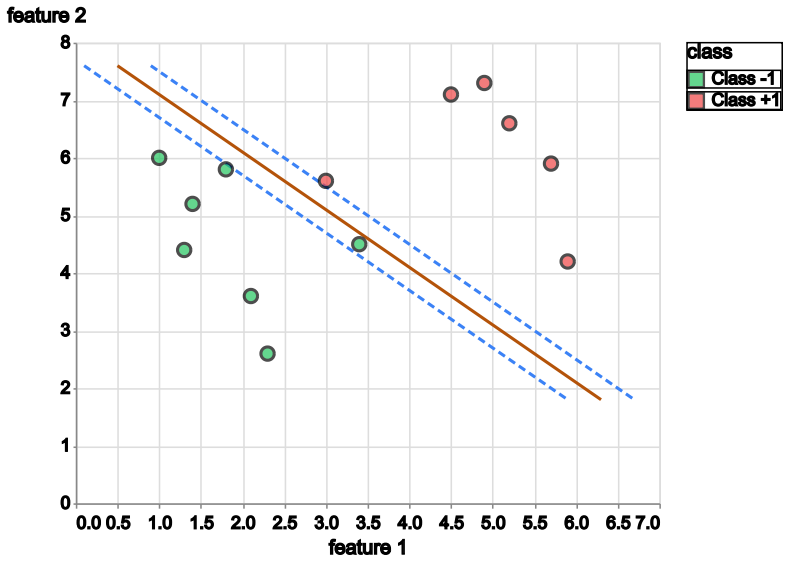
Such points are called the **support vectors** — they are the points that are literally "supporting" the maximum-margin classification region. Every other data point, sitting further away, plays no active role in determining the exact position of the optimal hyperplane once the support vectors are known.



The two circled points above sit exactly on the dashed margin lines — these are the support vectors for this particular dataset. Notice how removing any of the *other* (non-circled) points would not change the optimal hyperplane at all, since they never appear as tight constraints.

10. When Classes Refuse to Separate: Slack Variables

Everything so far assumed the two classes are **linearly separable**. But real datasets are rarely so accommodating. Consider a case where a handful of points genuinely violate the clean separation — for instance, one negative-class point that ends up mixed in among the positive class, and vice versa. In such a scenario, **no** $(\boldsymbol{\theta}, b)$ can satisfy $y_i(\boldsymbol{\theta}^\top \mathbf{x}_i + b) \geq 1$ for *every single* data point.



In the plot above, one green point (`3.4, 4.5`) has drifted onto the red side, and one red point (`3.0, 5.6`) has drifted onto the green side. If the previously-derived orange hyperplane is used, these two points end up on the *wrong* side of the classifier.

Should the whole idea of SVM be abandoned here? Not at all — in practice, only a **small number** of data points typically violate the separability condition, while the vast majority still sit comfortably on their correct side. Rather than demanding perfection from every point, the constraint can simply be **relaxed** for the problematic ones.

Introducing the slack variable. For each data point i, a non-negative slack variable $\eta_i \geq 0$ is introduced, which measures how much that particular point is allowed to violate the margin constraint:

$$\begin{aligned} \min_{\boldsymbol{\theta}, b, \boldsymbol{\eta}} \quad & \boldsymbol{\theta}^\top \boldsymbol{\theta} + C \sum_i \eta_i \\ \text{such that} \quad & y_i(\boldsymbol{\theta}^\top \mathbf{x}_i + b) \geq 1 - \eta_i \\ & \eta_i \geq 0 \end{aligned}$$

Here, $\eta_i = 0$ for any point that already satisfies the original hard constraint, while $\eta_i > 0$ allows that specific point to fall inside the margin (or even on the wrong side of the hyperplane) by an amount proportional to η_i. The constant $C > 0$ controls how harshly these violations are penalized in the objective — allowing *some* slack is fine, but allowing *unlimited* slack defeats the purpose, so the sum of slacks is added back into the cost function as a penalty.

Combining the two constraints into one expression. Both inequalities above can be merged into a single unconstrained-looking penalty term. Whenever $y_i(\boldsymbol{\theta}^\top \mathbf{x}_i + b) \geq 1$, no slack is needed at all ($\eta_i = 0$ is optimal); whenever it falls short, the required slack is exactly $\eta_i = 1 - y_i(\boldsymbol{\theta}^\top \mathbf{x}_i + b)$. This is precisely captured by a **hinge function**:

$$\min_{\boldsymbol{\theta}, b, \boldsymbol{\eta}} \quad \boldsymbol{\theta}^\top \boldsymbol{\theta} + C \sum_i \max\{0,\ 1 - y_i(\boldsymbol{\theta}^\top \mathbf{x}_i + b)\}$$

This single expression elegantly folds the margin-maximization objective, the classification constraint, and the slack penalty into one unconstrained cost function — the deeper justification for why this combination is valid will be explored further in the live sessions.

> **Note for Review:** The constant C is essentially a **regularization knob**. A very large C forces the model to punish margin violations heavily, pushing it toward the hard-margin behavior (small margin, few/no violations, but sensitive to noise). A very small C tolerates more violations in exchange for a wider, more robust margin. This trade-off is exactly analogous to bias–variance trade-offs seen elsewhere in ML.

11. Rewriting Everything as a Single Matrix Problem

Having derived the (hard-margin) optimization problem

$$\min_{\boldsymbol{\theta}, b} \boldsymbol{\theta}^\top \boldsymbol{\theta} \qquad \text{s.t.} \qquad y_i(\boldsymbol{\theta}^\top \mathbf{x}_i + b) \geq 1, \quad i = 1, \dots, n$$

the next task is to actually *solve* it. Since the cost function $\boldsymbol{\theta}^\top \boldsymbol{\theta}$ only involves $\boldsymbol{\theta}$ while the minimization is over both $\boldsymbol{\theta}$ and b, it helps to first bring b explicitly into the objective. This can be done for free by adding a $0 \times b^\top b$ term:

$$\boldsymbol{\theta}^\top \boldsymbol{\theta} + 0 \cdot b^\top b$$

which changes nothing numerically but makes the notation symmetric in $\boldsymbol{\theta}$ and b.

Unpacking the constraints one at a time. Writing out the first constraint ($i = 1$) fully:

$$y_1(\mathbf{x}_1^\top \boldsymbol{\theta} + b) \geq 1 \quad \Longrightarrow \quad y_1 \mathbf{x}_1^\top \boldsymbol{\theta} + y_1 b \geq 1$$

(note that y_1 is a scalar, so no transpose is needed on it.) Similarly for $i = 2$:

$$y_2 \mathbf{x}_2^\top \boldsymbol{\theta} + y_2 b \geq 1$$

and so on, up to the n-th point:

$$y_n \mathbf{x}_n^\top \boldsymbol{\theta} + y_n b \geq 1$$

Spotting the pattern. In every one of these n constraints, the *unknown* quantities are always $\boldsymbol{\theta}$ and b — identical across every row. What changes from row to row are the *known* quantities: $y_i \mathbf{x}_i^\top$ and y_i, both of which are fully determined by the training data. This means all n constraints can be stacked into a **single matrix inequality**.

Define the stacked parameter vector:

$$\boldsymbol{\omega} = \begin{bmatrix} \boldsymbol{\theta} \\ b \end{bmatrix}$$

and the data matrix A, where row i packs together the known quantities for constraint i:

$$A = \begin{bmatrix} y_1 \mathbf{x}_1^\top & y_1 \\ y_2 \mathbf{x}_2^\top & y_2 \\ \vdots & \vdots \\ y_n \mathbf{x}_n^\top & y_n \end{bmatrix}$$

You can verify that multiplying row i of A with the column vector $\boldsymbol{\omega}$ reproduces exactly $y_i \mathbf{x}_i^\top \boldsymbol{\theta} + y_i b$ — the left-hand side of constraint i. So **all n constraints collapse into one matrix inequality**:

$$A\,\boldsymbol{\omega} \geq \mathbf{1}$$

where $\mathbf{1}$ is a vector of all ones, of size $n \times 1$ (bold-face 1 to emphasize it is a *vector*, not the scalar one).

Rewriting the objective in terms of $\boldsymbol{\omega}$. The augmented cost function $\boldsymbol{\theta}^\top \boldsymbol{\theta} + 0 \cdot b^\top b$ can itself be written as a single quadratic form in $\boldsymbol{\omega}$:

$$\boldsymbol{\theta}^\top \boldsymbol{\theta} + b^\top \mathbf{0}\, b = \begin{bmatrix} \boldsymbol{\theta}^\top & b^\top \end{bmatrix} \begin{bmatrix} I & 0 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} \boldsymbol{\theta} \\ b \end{bmatrix} = \boldsymbol{\omega}^\top B\, \boldsymbol{\omega}, \qquad B = \begin{bmatrix} I & 0 \\ 0 & 0 \end{bmatrix}$$

Here I is the identity matrix matching the dimension of $\boldsymbol{\theta}$, so that B simply "switches off" the b-component while leaving the $\boldsymbol{\theta}$-component untouched — exactly reproducing $\boldsymbol{\theta}^\top \boldsymbol{\theta}$.

The final matrix-form optimization problem:

$$\min_{\boldsymbol{\omega}} \quad \boldsymbol{\omega}^\top B\, \boldsymbol{\omega} \qquad \text{such that} \qquad A\,\boldsymbol{\omega} \geq \mathbf{1}$$

This is now recognizable as a very standard class of optimization problem: the objective is **quadratic** in $\boldsymbol{\omega}$, and the constraint is **linear**. This makes it a **linearly-constrained quadratic optimization problem** — a Quadratic Program (QP) with inequality constraints. This is a well-studied class of problems, and solving it (for inequality-constrained problems in general) will be the subject of upcoming lectures.

12. A Warm-Up: Solving an Equality-Constrained Problem

Before tackling the inequality-constrained QP above, it's useful to build intuition with a much simpler *equality*-constrained problem — the SVM's inequality version will build on the exact same technique, extended appropriately.

The toy problem. Suppose the goal is:

$$\min_{\boldsymbol{\omega}} \quad \boldsymbol{\omega}^\top A\,\boldsymbol{\omega} \qquad \text{such that} \qquad \boldsymbol{\omega}^\top \boldsymbol{\omega} = 1$$

(assume A is symmetric). In words: minimize this quadratic form, but only among vectors $\boldsymbol{\omega}$ of **unit norm**.

Why not just minimize freely? If the constraint were ignored, the trivial minimizer would be $\boldsymbol{\omega} = \mathbf{0}$, since $\mathbf{0}^\top A\,\mathbf{0} = 0$ is the smallest possible value a quadratic form of this type can take (assuming A positive semi-definite, or simply the global minimum in this toy setting). But the zero vector clearly fails $\boldsymbol{\omega}^\top \boldsymbol{\omega} = 1$ — so unconstrained minimization is not automatically compatible with the constraint. The constraint has to be *actively enforced*.

The penalty (Lagrange multiplier) approach. A penalty term is introduced that equals zero whenever the constraint is satisfied, and grows whenever $\boldsymbol{\omega}$ deviates from it:

$$L(\boldsymbol{\omega}, \lambda) = \boldsymbol{\omega}^\top A\,\boldsymbol{\omega} - \lambda\,(\boldsymbol{\omega}^\top \boldsymbol{\omega} - 1)$$

Taking the gradient with respect to $\boldsymbol{\omega}$ and setting it to zero (using A symmetric, so $\nabla_{\boldsymbol{\omega}}(\boldsymbol{\omega}^\top A\boldsymbol{\omega}) = 2A\boldsymbol{\omega}$):

$$2A\boldsymbol{\omega} - 2\lambda\boldsymbol{\omega} = 0 \quad \Longrightarrow \quad A\boldsymbol{\omega} = \lambda\boldsymbol{\omega}$$

This is nothing but the **standard eigenvalue–eigenvector equation**: $\boldsymbol{\omega}$ must be an eigenvector of A, and λ the corresponding eigenvalue.

Connecting back to the objective. Multiplying both sides of $A\boldsymbol{\omega} = \lambda\boldsymbol{\omega}$ on the left by $\boldsymbol{\omega}^\top$:

$$\boldsymbol{\omega}^\top A\boldsymbol{\omega} = \lambda\,\boldsymbol{\omega}^\top \boldsymbol{\omega} = \lambda \quad (\text{since } \boldsymbol{\omega}^\top \boldsymbol{\omega} = 1)$$

So the value of the objective function *at any eigenvector solution* is simply the corresponding **eigenvalue** λ. Since the goal is to *minimize* $\boldsymbol{\omega}^\top A\boldsymbol{\omega}$, the optimal $\boldsymbol{\omega}$ must be the eigenvector of A corresponding to its **smallest eigenvalue**.

> **Note for Review:** This technique — introducing a penalty term proportional to how much a constraint is violated, then differentiating — is the essence of the **Lagrange multiplier method**. Here it was applied to an equality constraint, producing an elegant eigenvalue problem. The SVM's actual constraints are *inequalities*, not equalities, so this exact eigen-trick does not directly transfer; a more general framework (Lagrangian duality with KKT conditions) is needed, which builds on this same core idea and will be covered in upcoming lectures.

13. Think-It-Through Questions

(These are meant to be reasoned through, not looked up. Take your time.)

1. Suppose a 2D, linearly separable dataset produces an optimal SVM hyperplane with exactly one support vector on each side. If every non-support-vector point is now deleted from the dataset and the SVM optimization problem is solved again on this smaller dataset, will the learned hyperplane change? Justify your answer using the structure of the constraint set $A\boldsymbol{\omega} \geq \mathbf{1}$.

2. The derivation in Section 8 rescales $(\boldsymbol{\theta}, b)$ so that the closest point satisfies $\min_i |\boldsymbol{\theta}^\top \mathbf{x}_i + b| = 1$, rather than leaving this minimum value as a free variable to optimize over. Explain concretely what would go wrong (or what redundancy would appear) if c in $\min_i |\boldsymbol{\theta}^\top \mathbf{x}_i + b| = c$ were kept as an extra decision variable inside the optimization problem, alongside $\boldsymbol{\theta}$ and b.

3. In the soft-margin formulation $\min_{\theta,b,\boldsymbol{\eta}} \ \boldsymbol{\theta}^\top \boldsymbol{\theta} + C \sum_i \eta_i$, describe qualitatively what happens to (a) the width of the margin and (b) the number of points allowed to violate the margin, as $C \to \infty$ versus as $C \to 0$. Based on this, explain in your own words why C behaves like a regularization parameter, and which regime is more prone to overfitting.

4. The toy warm-up problem in Section 12 was solved using an eigenvector trick because its constraint was an **equality** ($\boldsymbol{\omega}^\top \boldsymbol{\omega} = 1$). The actual SVM problem has n **inequality** constraints ($A\boldsymbol{\omega} \geq \mathbf{1}$) instead. Why can't the equation $A\boldsymbol{\omega} = \lambda \boldsymbol{\omega}$ be reused directly for the SVM problem? (Hint: think about what it would mean for *all* n inequality constraints to become tight/active simultaneously, and whether that is generally possible or even desirable.)

5. In a soft-margin SVM, is it possible for a data point that lies **strictly inside** the margin — i.e., one that violates $y_i(\boldsymbol{\theta}^\top \mathbf{x}_i + b) \geq 1$ and therefore requires $\eta_i > 0$ — to *not* end up being called a "support vector," even though the hard-margin definition of a support vector only covers points with $y_i(\boldsymbol{\theta}^\top \mathbf{x}_i + b) = 1$ exactly? Think about how the definition of a support vector needs to be extended once slack variables enter the picture, and justify your reasoning.

End of Lecture 5 notes — Support Vector Machines: geometric foundations, the maximum-margin formulation, support vectors, soft-margin relaxation via slack variables, and the matrix-quadratic-program view of the optimization problem.